

TinyCore8: Um processador RISC 8-bits

Load/Store em Verilog

David Oliveira Silva
Salvador-BA, Brasil
david.oliveira@cepedi.org.br

Abstract— Este documento detalha o projeto, a implementação em Verilog e a verificação funcional de um processador RISC de 8 bits, denominado "TinyCore8". O processador implementa uma arquitetura Harvard e um conjunto de instruções Load/Store, operando sobre um banco de dois registradores (A e B). A Unidade de Controle foi implementada como uma Máquina de Estados Finitos (FSM) multi-ciclo para gerenciar o Caminho de Dados (Datapath). A verificação funcional foi conduzida no ModelSim, utilizando um programa de teste em uma linguagem customizada, inspirada em assembly, para validar as operações de ULA e o acesso à memória (RAM e ROM). O projeto demonstra um ciclo de desenvolvimento de hardware completo, da especificação à simulação, resultando em um core de CPU funcional e verificável.

Keywords— Processador, Verilog, RISC, Arquitetura Load/Store, FPGA, Unidade de Controle, Datapath, FSM.

I. INTRODUÇÃO

Este documento detalha o projeto e a implementação de um processador RISC (Reduced Instruction Set Computer) de 8 bits, o "TinyCore8". O objetivo principal foi a aplicação prática dos conceitos estudados sobre a linguagem de descrição de hardware em uma arquitetura de computadores através da construção de um core de CPU funcional.

A arquitetura segue o modelo Load/Store, onde operações aritméticas e lógicas ocorrem exclusivamente entre registradores. O acesso à memória é realizado por instruções de carga (Load) e armazenamento (Store) dedicadas. Para otimizar a busca de instruções, o processador implementa uma Arquitetura Harvard, utilizando memórias separadas para instruções (ROM) e dados (RAM), ambas acessadas por barramentos de 8 bits.

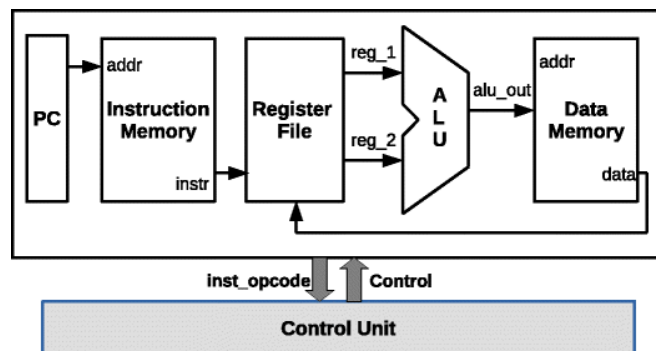


Fig. 1. Diagrama de bloco de um processador RISC com pipeline de cinco estágios.

O desenvolvimento foi realizado em Verilog, com foco na clara separação entre o Caminho de Dados (Datapath) e a Unidade de Controle (Control Unit).

II. ARQUITETURA E DESENVOLVIMENTO

O processador é composto por dois blocos principais: o Datapath, que contém os elementos de processamento e armazenamento, e a Control Unit, que decodifica instruções e orquestra o Datapath. A tabela 1 apresenta todas as instruções que foram implementadas:

Tabela 1. Todas as instruções do TinyCore8, exigidas e adicionais.

MNE.	OP.	SINTAXE	DESCRIÇÃO	CICLOS
ADD	0000	ADD #addr	$RAM[addr] \leftarrow A + B$	4
SUB	0001	SUB #addr	$RAM[addr] \leftarrow A - B$	4
LDA	0010	LDA #addr	$A \leftarrow RAM[addr]$	4
STA	0011	STA #addr	$RAM[addr] \leftarrow A$	3
LDB	0100	LDB #addr	$B \leftarrow RAM[addr]$	4
STB	0101	STB #addr	$RAM[addr] \leftarrow B$	3
LDC	0110	LDC #imm	$A \leftarrow imm$	3
JMP	0111	JMP #addr	Se $A == 0$ então $PC \leftarrow addr$	4
AND	1000	AND #addr	$RAM[addr] \leftarrow A \wedge B$	4
OR	1001	OR #addr	$RAM[addr] \leftarrow A \vee B$	4
XOR	1010	XOR #addr	$RAM[addr] \leftarrow A \oplus B$	4
NOT	1011	NOT #addr	$RAM[addr] \leftarrow \sim A$	4
MUL	1100	MUL #addr	$RAM[addr] \leftarrow (A \times B)_{(7:0)}$	4
DIV	1101	DIV #addr	$RAM[addr] \leftarrow A / B$	4
CMP	1110	CMP #addr	$RAM[addr] \leftarrow Compare(A, B)^*$	4
LDD	1111	LDD #imm	$B \leftarrow imm$	3

$Compare(A, B)^*$: Retorna 0x01 se $A > B$, 0xFF se $A < B$ e 0x00 quando $A == B$.

A. Formato da Instrução

Para simplificar, foi definido um formato de instrução fixo de 12 bits. Os 4 bits mais significativos (MSB) representam o opcode (ADD, SUB etc.) e os 8 bits menos significativos

(LSB) representam o operando (um valor imediato ou um endereço de memória).

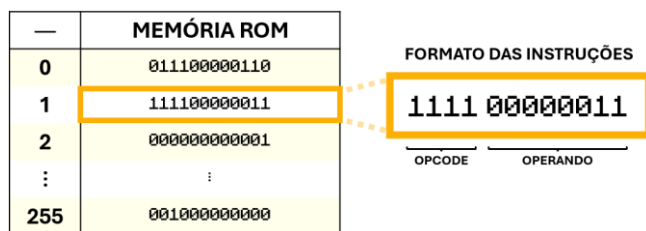


Fig. 2. Estrutura de armazenamento na memória de programa e detalhamento do formato da instrução de máquina.

B. Caminho de Dados

O caminho de dados ou Datapath (Datapath.v) agrupa os blocos funcionais que processam os dados. Assim, temos:

1) *Contador de Programa*: Trata-se de registrador de 8 bits que armazena o endereço da próxima instrução. Possui lógica para incrementar (pcIncEnable) ou carregar um novo endereço (pcLoadEnable), essencial para a instrução JMP. Na implementação em verilog pode ser encontrado como “ProgramCounter.v”.

2) *Bando de Registradores*: Contém os dois registradores de 8 bits especificados, A e B, com entradas de habilitação de escrita (writeEnableA, writeEnableB) independentes. Pode ser encontrado no arquivo “RegisterBank.v”.

3) *Unidade Lógica e Aritmética (ULA)*: Módulo combinacional que implementa as operações de 4 bits definidas (como ADD, SUB, AND e CMP). As outras operações, como armazenamento e o próprio JMP, são realizadas diretamente na unidade de controle. A implementação pode ser localizada nos arquivos do projeto como “ArithmeticLogicUnit.v”

4) *Lógica de Roteamento (MUXes)*: A implementação “Datapath.v” utiliza multiplexadores para direcionar corretamente o fluxo de dados. Assim, há um MUX na entrada de cada registrador, responsável por selecionar a fonte dos valores (RAM ou operando imediato). Além disso, existe um MUX na segunda entrada da ULA (aluIn2MuxSel), que escolhe entre o registrador B e uma constante zero. Por fim, há um MUX na entrada da memória de dados (ramInMuxSel), que define qual fonte será usada na escrita (saída da ULA, registrador A ou registrador B).

C. Unidade de Controle (FSM)

A Unidade de Controle (ControlUnit.v) é uma Máquina de Estados Finitos (FSM) que gera todos os sinais de controle do Datapath. Ela opera em um ciclo fundamental de busca, decodificação e execução. A figura 3 exibe o diagrama de estados adotado, mas também é possível consultar através da [plataforma mermaid](#) com mais qualidade.

1) *ESTADO FETCH*: Este é o estado imediatamente posterior ao término do reset. O Program Counter (PC) fornece o endereço da próxima instrução para a ProgramROM, que disponibiliza seu conteúdo no barramento de instruções. A função do estado FETCH é exclusivamente colocar a instrução correspondente ao PC no registrador de entrada da Control Unit para que, no ciclo seguinte, possa ser decodificada. Por isso, o próximo estado é sempre o DECODE.

2) *ESTADO DECODE*: A instrução recém-capturada no estado FETCH é analisada e o opcode é identificado, permitindo que a FSM determine qual será o próximo estado de execução. Nesse momento, o opcode é armazenado em um registrador interno (instructionRegister ou IR). Esse registrador mantém o opcode estável durante todos os ciclos necessários para completar a instrução atual, evitando conflitos com instruções subsequentes que possam ser carregadas durante o avanço do PC.

3) *ESTADOS DE EXECUÇÃO*: Existem duas possibilidades nesse caso, assim teremos:

a) *Ciclo Único (LDC, STA etc.)*: Instruções como LDC são executadas em um único estado (EXEC_LDC), que ativa os enables e MUXes corretos e, ao final, o pcIncEnable para incrementar o endereço da memória de programa.

b) *Multi-Ciclo (LDA, ADD, JMP etc.)*: Instruções que dependem de um resultado da RAM ou da ULA demoram mais ciclos. A instrução LDA passa pelos estados EXEC_LDA, onde o endereço é enviado à RAM, e WRITE_REGA, onde o dado retornado da RAM é capturado e escrito no registrador A. A instrução ADD percorre EXEC_ALU, que configura a ULA para realizar a operação aritmética, e em seguida WRITE_RAM, onde o valor produzido em aluResult é escrito na RAM. Já a instrução JMP passa por EXEC_JMP, que prepara a ULA para comparar o registrador A com zero, e depois por EXEC_JMP_2, onde o resultado dessa comparação é analisado para decidir entre ativar pcLoadEnable (realizando o salto) ou pcIncEnable (seguindo para a próxima instrução).

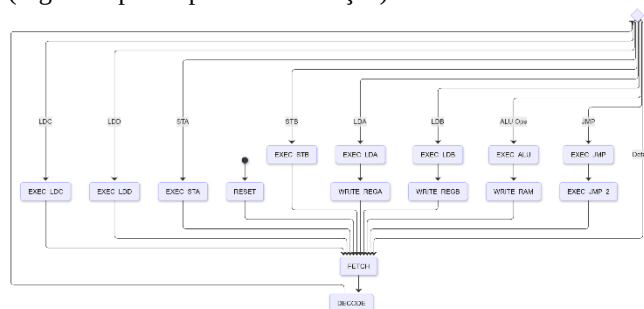


Fig. 3. Diagrama de estados da FSM implementada para controlar o Datapath, onde o opcode define a transição do estado decode.

III. RESULTADOS E VERIFICAÇÃO

A verificação do processador foi realizada através de simulação no ModelSim, utilizando um testbench (TinyCore8TB.v) e um programa com uma pequena sequência de instruções. Para evitar a programação direta na memória de instruções (ROM), um algoritmo em Python foi elaborado para realizar a montagem das instruções em texto para linguagem de máquina em código binário, veremos mais a seguir.

A. Programa de Teste

Para validar a funcionalidade, foi desenvolvido um programa em um arquivo de texto (.txt) em uma linguagem de montagem customizada, focado em testar o ciclo completo de leitura e escrita na memória de dados. Não foi utilizado todas as instruções dentro do programa de teste, embora isso seja possível.



Fig. 4. Arquivo de texto contendo o programa de teste para validação funcional do processador em Verilog.

B. Ferramenta de Montagem (TXTtoMIF)

Para traduzir a linguagem de montagem (legível por humanos) para o formato de inicialização de memória (.mif) exigido pelo IP da ProgramROM do Quartus, foi desenvolvido um tradutor de instruções em Python. Como o foco deste trabalho é a descrição do hardware, não será abordado muitos detalhes da implementação em alto nível do tradutor.

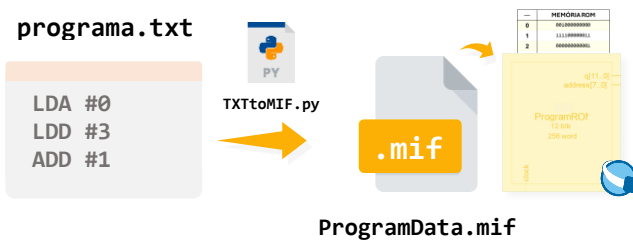


Fig. 5. Conversão do arquivo de texto para o formato de inicialização de memória (.mif) através do Python.

De modo geral, o script realiza a leitura do arquivo “programa.txt” e executa a análise sintática de cada linha, identificando e separando o mnemônico de seu respectivo operando. Em seguida, procede à tradução do mnemônico para o opcode binário de 4 bits definido em “Definitions.v”, bem como à conversão do operando em uma representação binária de 8 bits. Após essas etapas, o script combina ambos os campos para compor a instrução final de 12 bits e gera o arquivo “ProgramData.mif”, que foi utilizado na inicialização da IP da memória de programa.

C. Análise da Simulação

A simulação no ModelSim foi executada através de um script (sim.do), que compila as bibliotecas de IP da Intel e configura a geração automática das formas de onda no software da Intel. É importante destacar que a memória de dados (RAM) iniciou a simulação apenas com o primeiro endereço preenchido com o valor 1, o restante recebeu 0. Os

resultados confirmam a operação correta do processador. A análise da forma de onda (waveform) permitiu observar:

Conforme a figura 7, no início da execução após o reset, a instrução LDA #00 é decodificada, fazendo o sinal state transitar por EXEC_LDA e WRITE_REGA. A forma de onda mostra o registrador REGA sendo atualizado para o valor 1, evidenciando que o processador leu corretamente o dado pré-existente na RAM. Em seguida, a instrução LDD #03 carrega o valor imediato 3 no registrador REGB.

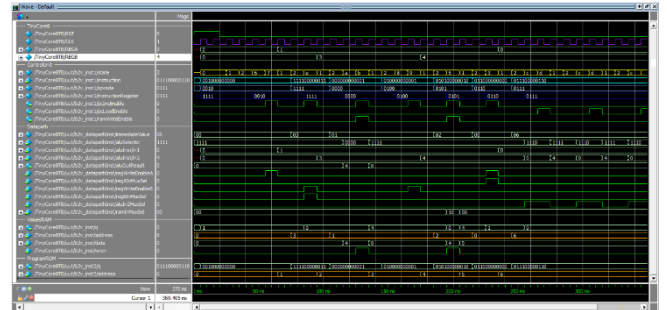


Fig. 6. Simulação completa do programa de teste rodado no processador TinyCore8 no ModelSim.

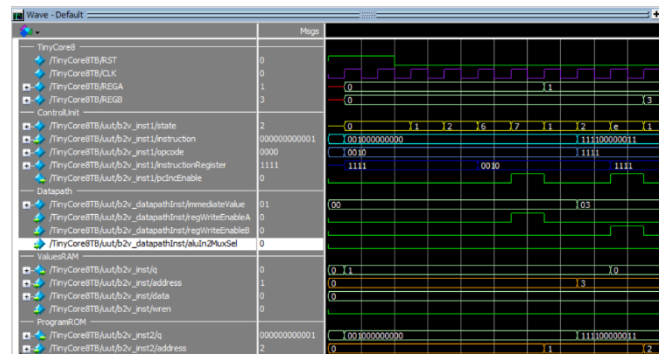


Fig. 7. Simulação no ModelSim das instruções LDA e LDD, armazenando valores nos registradores A e B.

Na etapa seguinte ocorre a instrução ADD #01, que realiza a soma entre os registradores, resultando em 4. A máquina de estados direciona esse resultado para escrita na memória. A forma de onda exibe o valor 4 na entrada de dados e o endereço 01 da memória RAM, ambos sincronizados com o pulso de wren, confirmando que o resultado da ULA foi armazenado corretamente no endereço vindo do imediato (0x01).

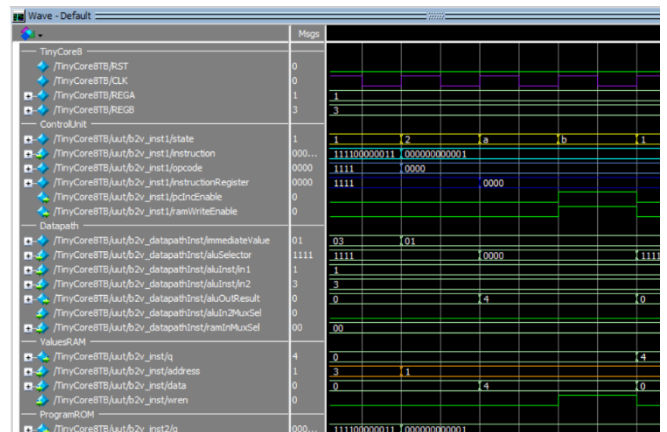


Fig. 8. Semelhante a anterior, mas considerando a instrução ADD.

A verificação de consistência ocorre com a instrução LDB #01, responsável por ler o endereço 0x01, onde o valor 4 havia sido recém-escrito. A atualização do registrador REGB de 3 para 4 demonstra que o ciclo completo — processamento, escrita na RAM e leitura subsequente — funcionou como esperado.

A instrução STB #02 realiza a movimentação do valor presente em REGB (4) para o endereço 0x02. O sinal de escrita da RAM (wren) é novamente ativado, com o barramento de endereços apontando para 0x02, validando a operação de armazenamento no segundo registrador de memória. Assim, a memória de dados deve apresentar o valor 4 armazenado duas vezes seguidas (ADD e STB).

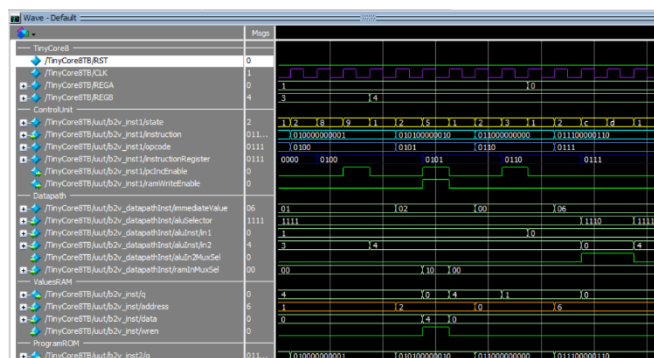


Fig. 9. Semelhante a anterior, mas considerando as instruções STB, LDC e JMP.

Por fim, a instrução LDC #00 zera o registrador A, preparando a condição de salto. A instrução JMP #06 verifica essa condição e o ProgramCounter passa a conter o endereço 06, estabelecendo o laço infinito de encerramento da execução.

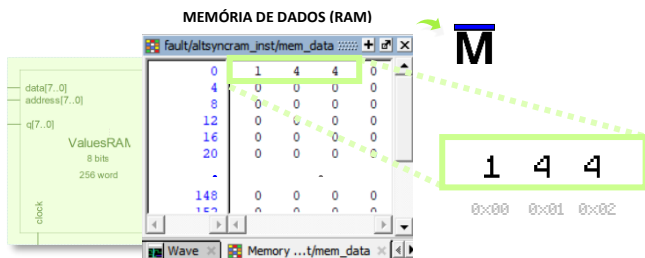


Fig. 10. Memória RAM no ModelSim ao final da simulação, indicando que o valor 4 foi salvo corretamente.

IV. CONCLUSÃO

O projeto atingiu seu objetivo principal: um processador de 8 bits, com arquitetura Load/Store, foi implementado com sucesso em Verilog e verificado funcionalmente.

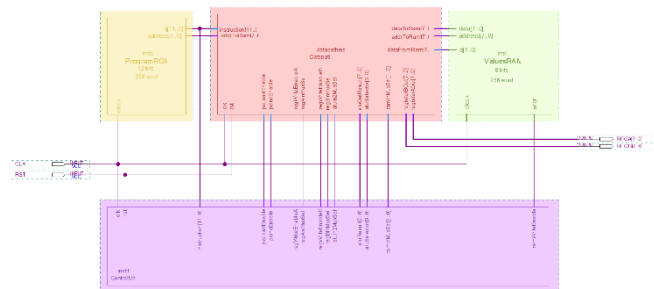


Fig. 11. Diagrama de blocos do TinyCore8, obtido no Quartus Prime (TinyCore8.bdf).

O desenvolvimento demonstrou a complexa, porém vital, interação entre o Caminho de Dados e a Unidade de Controle. A criação de um montador customizado (TXTtoMIF.py) permitiu uma verificação de software robusta.

O programa de validação contemplou instruções representativas de todas as classes funcionais da arquitetura: Aritmética, Lógica, Acesso à Memória e Controle de Fluxo. Embora o teste não tenha exaurido todo o conjunto de instruções, a verificação deste subconjunto crítico, aliada à regularidade do Caminho de Dados (onde instruções da mesma categoria compartilham recursos de hardware similares), fornece evidências sólidas da integridade do projeto. Adicionalmente, a infraestrutura de software desenvolvida, em especial o montador customizado em Python, permite a modificação ágil da memória de programa, viabilizando a expansão da cobertura de testes para as demais instruções conforme a necessidade de validações futuras.

Como trabalhos futuros, o processador pode ser expandido para incluir um stack pointer (ponteiro de pilha) para chamadas de sub-rotinas (CALL/RET), a implementação de uma arquitetura pipelined e a expansão das instruções implementadas como uma para indicar o encerramento de programa.

V. REFERÊNCIAS

- [1] CI DIGITAL. . Projeto final: construção de um processador de 8 bits em Verilog. [S.l.]: Programa de Desenvolvimento de Competências em Sistemas Digitais, 2025. 3 p.
- [2] Francisquini, R. Projetando um Processador Simples em Verilog. Medium, 22 jun. 2018. Disponível em: <https://medium.com/@francisquini/projetando-um-processador-simples-em-verilog-ea1b67f36da2>. Acesso em: 17 nov. 2025.
- [3] Pereira, R. Processadores Programáveis – Como projetar um processador em Verilog: Arquitetura – parte 1. Embarcados, 18 fev. 2015. Disponível em: <https://embarcados.com.br/processadores-programaveis-verilog-arquitetura-1/>. Acesso em: 17 nov. 2025.